

subfunction management using devlink

Parav Pandit, 8/18/2020

AGENDA

- User requirements
- Devlink overview and why devlink
- Design
- Devlink extensions and examples
- Open issues
- Questions

USER REQUIREMENTS

- Ability to create, configure, deploy virtual device from a NIC side
- Same user interface from NIC side or the host side
- Able to configure device before its discovered by OS and/or pre boot drivers
  - Such as mac address, rate, link status, policer, encap/decap
- Single PCI (PF/VF/SF) device supporting multiple class of devices at a time
  - netdev, rdma, vdpa and more
- Ability to map a ‘whole function’ to VM
- Use a device for VM/networking/bare-metal/container
- Ability to share low level resources with parent PCI device such as IRQ vectors, BAR region and more

NVIDIA.

USER REQUIREMENTS (CONTINUE…)

- Run user space DPDK and kernel driver for a given PF/VF/SF
- Use rich tc offloads interface for networking traffic of PF/VF/SF
- Ability to map switch representor with its ‘portion of the device’
- Deterministic device naming of rdma and netdevice of PF/VF/SF using systemd/udev
- Deterministic representor device naming

NVIDIA.

WHAT THESE REQUIREMENTS MEANS?

Sub function device

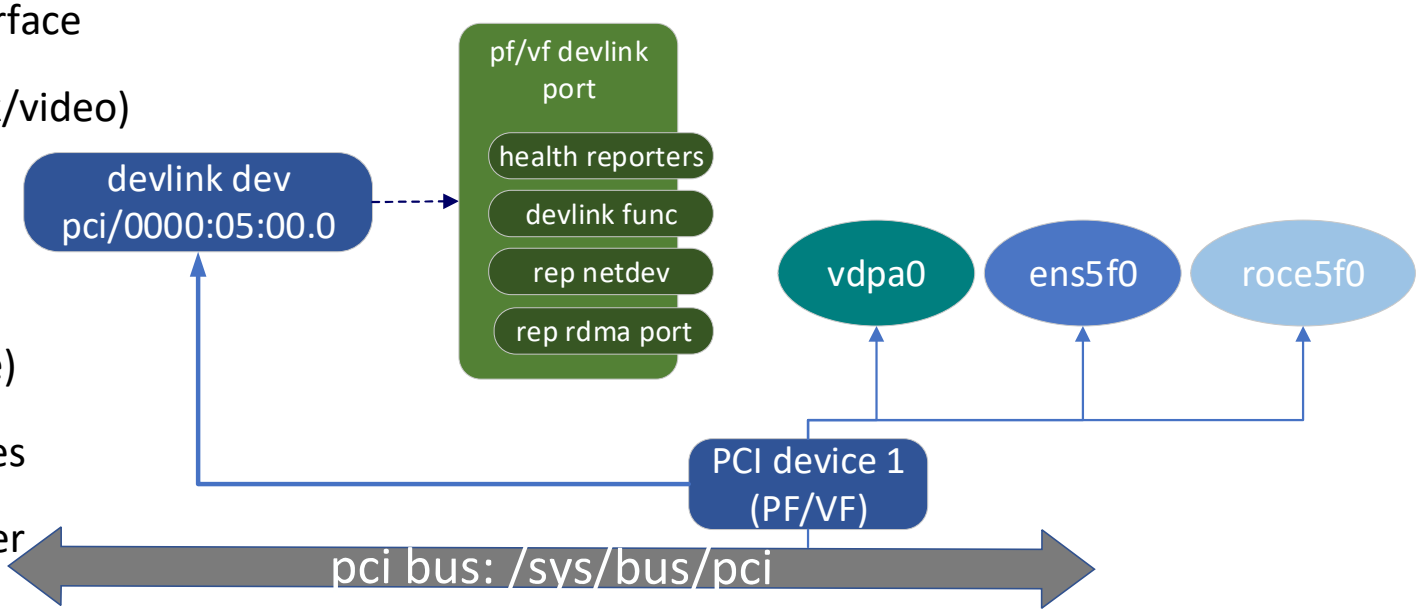
- User should be able to create ‘portion of the device’:
  - a new light weight PCI function and its associated class devices
  - One at a time, instead of bulk device enable/disable
- ‘portion of the device’ = sub function(SF) of a PCI device (= aka as ‘slice’/’vdev’ in past.)
- Sub function is like PCI SRIOV VF but light weight in nature.
- Subfunction doesn’t need SRIOV enablement.
- Subfunction shares resources with underlying PCI device
- Subfunction is a ‘struct device’ with its own BAR resources
  - lives on software bus with link to parent PCI device
  - Follows probe/remove/power management

NVIDIA CONFIDENTIAL. DO NOT DISTRIBUTE.

NVIDIA.

DEVLINK OVERVIEW

- PCI level device management interface
- Agnostic of device class (net/block/video)
- Health recovery, report
- Device parameter configuration
- Reload device (after config change)
- Used mainly for networking devices
- Net namespace aware and syscaller friendly

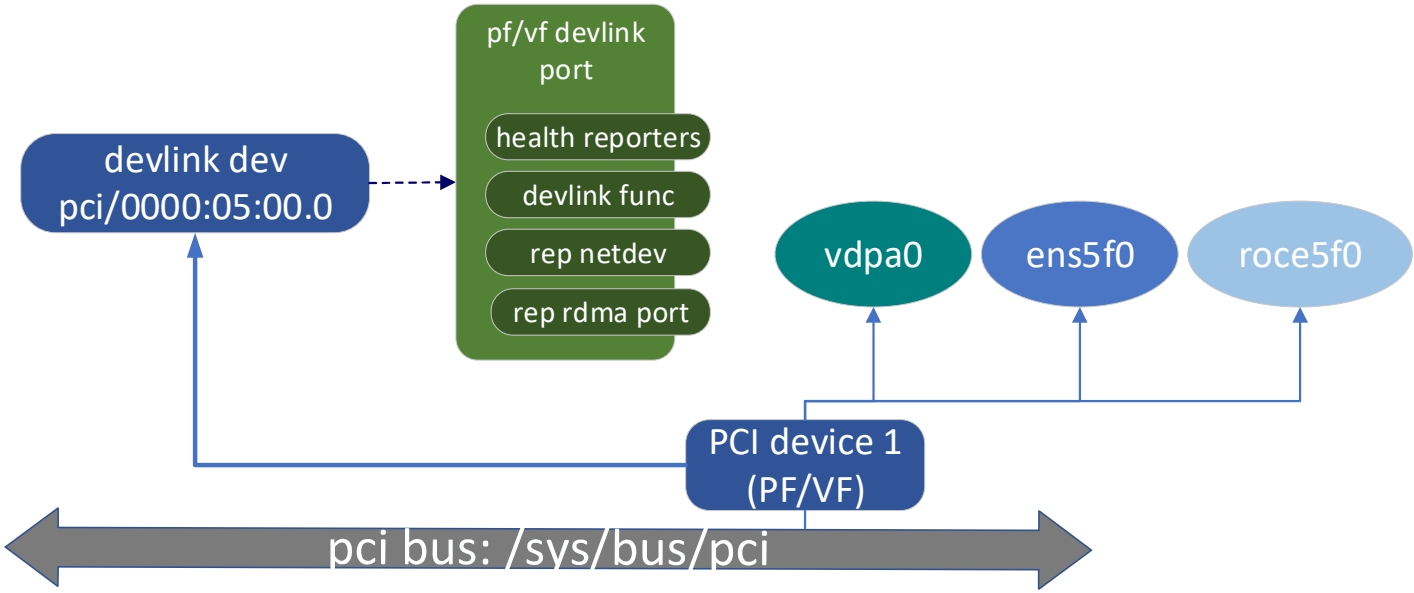


NVIDIA CONFIDENTIAL. DO NOT DISTRIBUTE.

NVIDIA.

DEVLINK OVERVIEW (CONTINUE…)

- Supports eswitch offloads configuration
- Connects eswitch ports with representors
- Eswitch IS OPTIONAL
- Works without eswitch

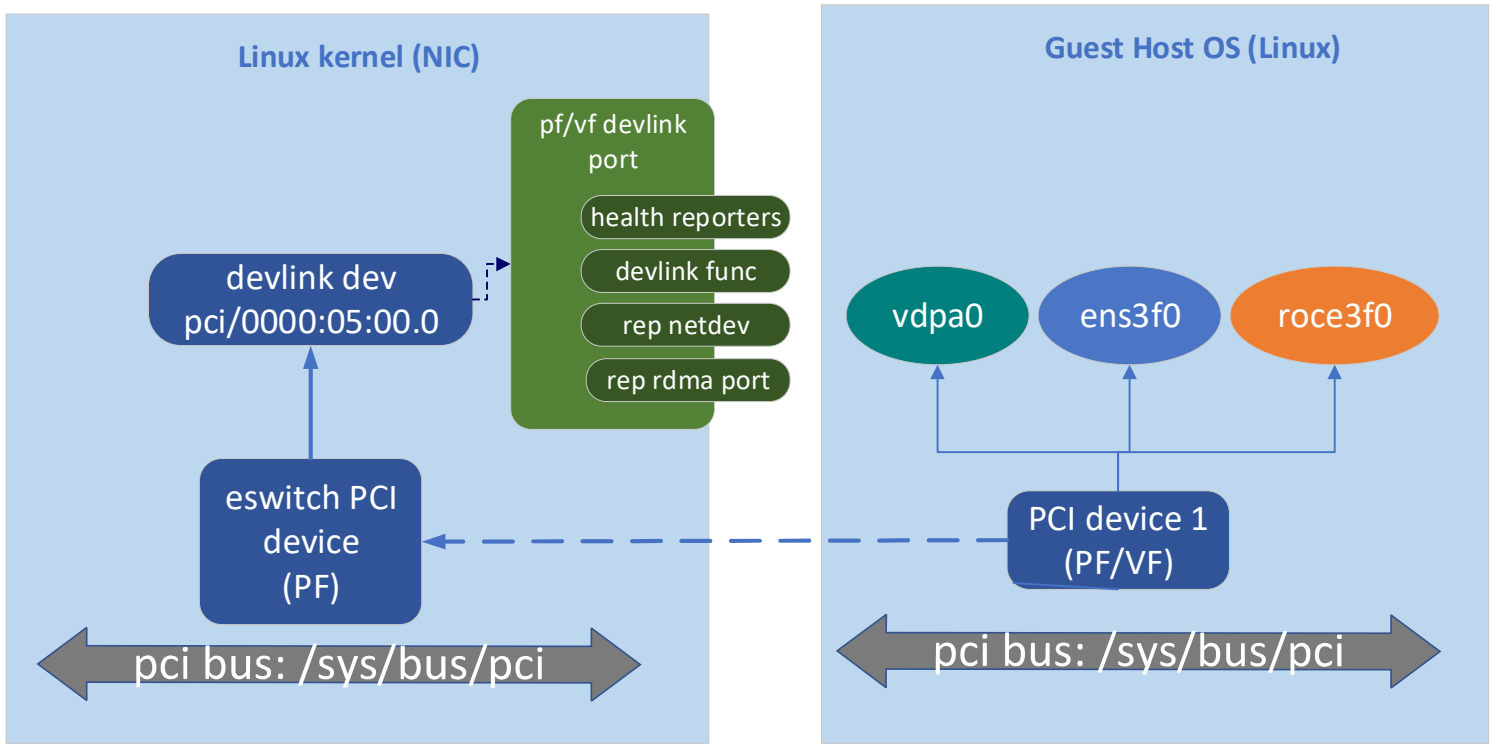


NVIDIA CONFIDENTIAL. DO NOT DISTRIBUTE.

NVIDIA.

CURRENT SPLIT NIC SW KERNEL STACK

- Aka smartnic
  - eswitch and actual PF/VF are running on on two different linux system
  - eswitch runs linux kernel, but host may be running other operating system



WHY DEVLINK FOR SUB FUNCTION MANAGEMENT

- ip link set <dev> vf <num> param doesn't fit the requirements
  - Inability to create, config, deploy model
  - Unable to create, config, deploy from NIC side
  - Unable to handle non-VF accelerated functions (SF)
  - Not suitable for non-networking devices
  - Misses linkage with representor switchdev model
- Why not sysfs?
  - Exposes in all net namespace, security concern
  - Inability to report error, warning strings in user commands (applies to configfs too)
  - inodes in addition to existing data structs for each config
- Why not configfs?
  - May be
  - Misses all netlink features
- Why not char device with ioctl?
  - Netlink is lot more mature features with traces, errors, net ns, TLV attributes, nested attributes,
  - Error code is not sufficient to convey an error. Extack is lot more informative to user.
  - Reinvents netlink!
- Why devlink?
  - Overcomes all the above limitations
  - Already mature

NVIDIA CONFIDENTIAL. DO NOT DISTRIBUTE.

NVIDIA.

USER FLOW USING DEVLINK

devlink port

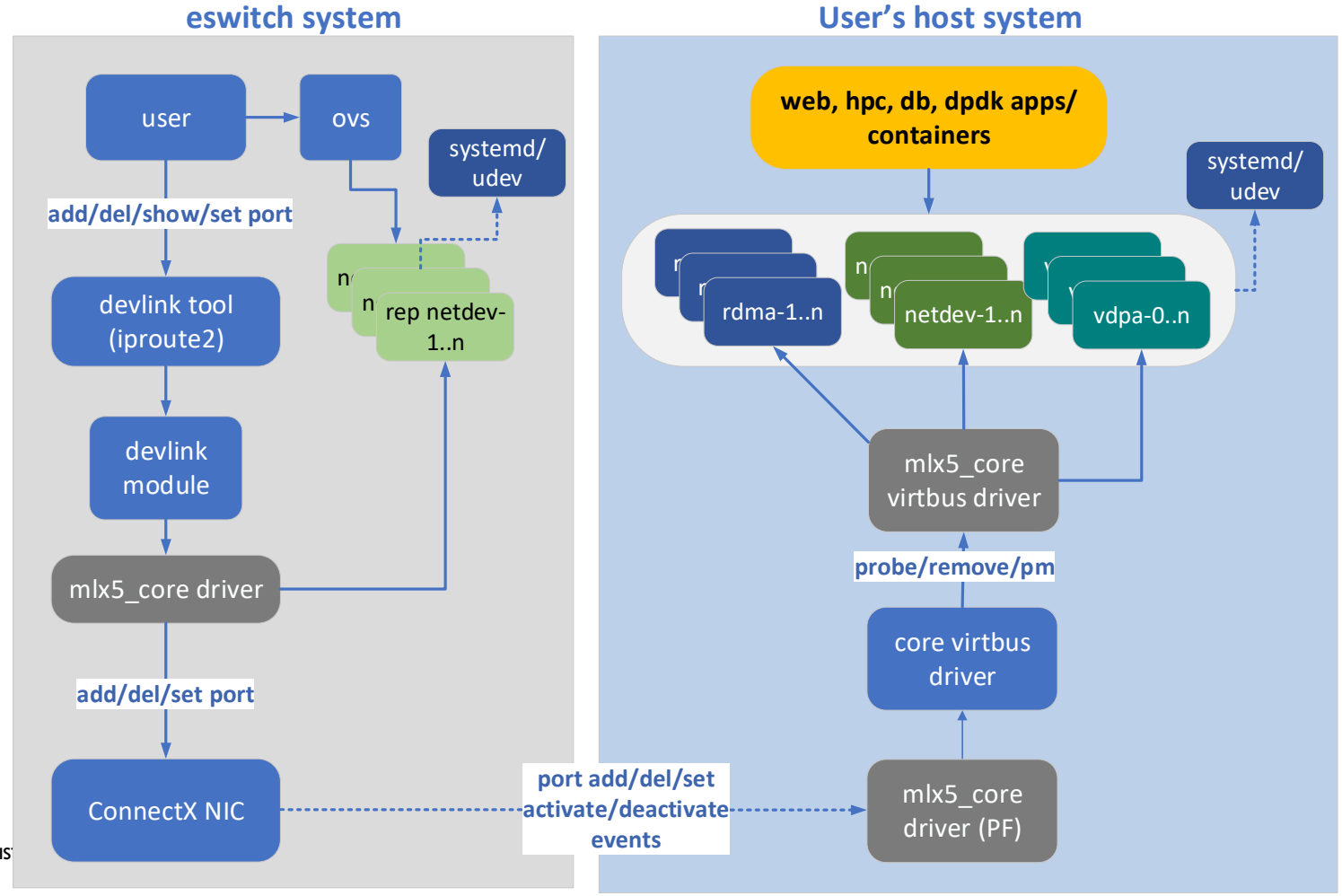


- Create, configure, deploy – all commands on eswitch system.
- Create/Delete
  - Create/Delete a devlink port
- Configure
  - Configure port, port function
  - Create and configuration on single system (eswitch host)
- Deploy
  - Deploy a device to local or external host
    - Decide at creation time.
  - Like soft hot plug/unplug a device.
  - Deploy after enough configuration is completed.

NVIDIA CONFIDENTIAL. DO NOT DISTRIBUTE.

NVIDIA.

SPLIT STACK DEEP DIVE



NVIDIA CONFIDENTIAL. DO NOT DIS

NVIDIA.

DEVLINK EXTENSION EXAMPLES

devlink port add/del/set/show (**eswitch** System)

(1) Port add:

```
$ devlink port add pci/0000:03:00.0 flavour pcisf pfnum 0 sfnum 46
```

(2) Port show:

```
$ devlink port show

 pci/0000:03:00.0/1 type eth netdev ens2f0pf0sf46 flavour pcisf pfnum 0 sfnum 46
```

(3) Port function configuration:

```
$ devlink port function set pci/0000:03:00.0 /1 hw_addr 00:11:22:33:44:55 state active
```

(4) Port delete: (delete by port index)

```
$ devlink port del pci/0000:03:00.0/1
```

NVIDIA CONFIDENTIAL. DO NOT DISTRIBUTE.

NVIDIA.

DEVLINK EXTENSION EXAMPLES

devlink port show (eswitch system)

Port show in JSON format:

```
$ devlink port show pci/0000:03:00.0/1 –jp
{
        “pci/0000:03:00.0/1": {
                "type": "eth“,
                "netdev": " ens3f0pf0sf46",
                "flavour": "pcisf",
                "pfnum": 0,
                "sfnum": 46,
                "function": {
                        "hw_addr": "00:11:22:33:44:55",
                        "state": "active",
                        “opstate": "attached",       /* more in later slides */

                }
          }
}
```

NVIDIA CONFIDENTIAL. DO NOT DISTRIBUTE.

DEVLINK EXTENSION EXAMPLES

devlink dev show, port add/show/del (guest/user host system)

(5) Device show:

$ devlink dev show

pci/0000:06:00.0
pci/0000:06:00.0%sf46

(6) Port show:

$ devlink port show

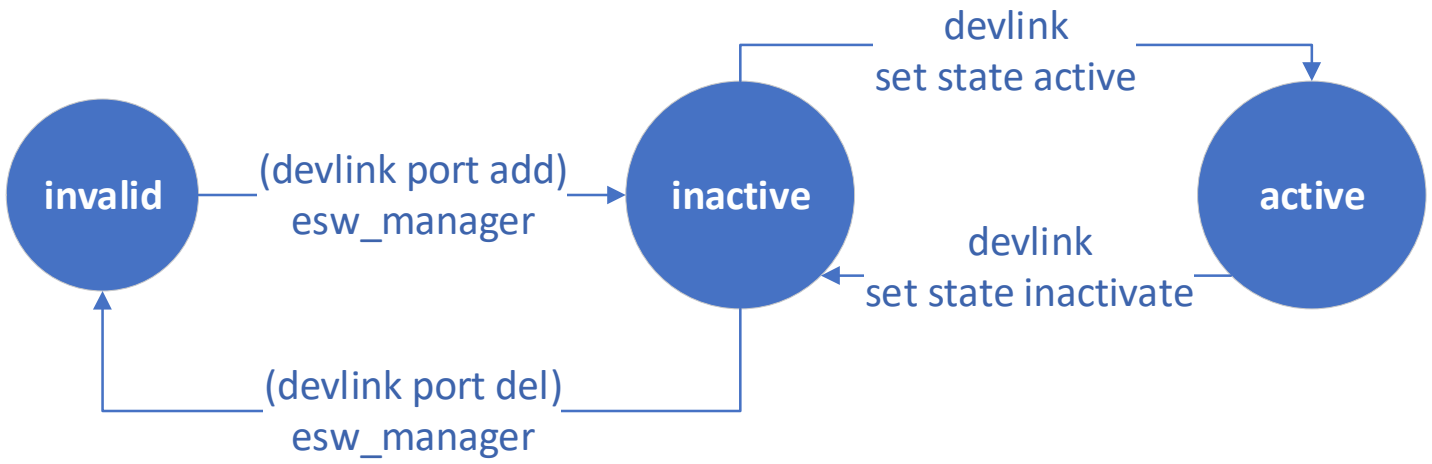
pci/0000:06:00.0%sf46/0: flavour virtual type eth netdev netdev enp3s0f0s46

NVIDIA CONFIDENTIAL. DO NOT DISTRIBUTE.

NVIDIA.

PORT FUNCTION STATES

port function state machine



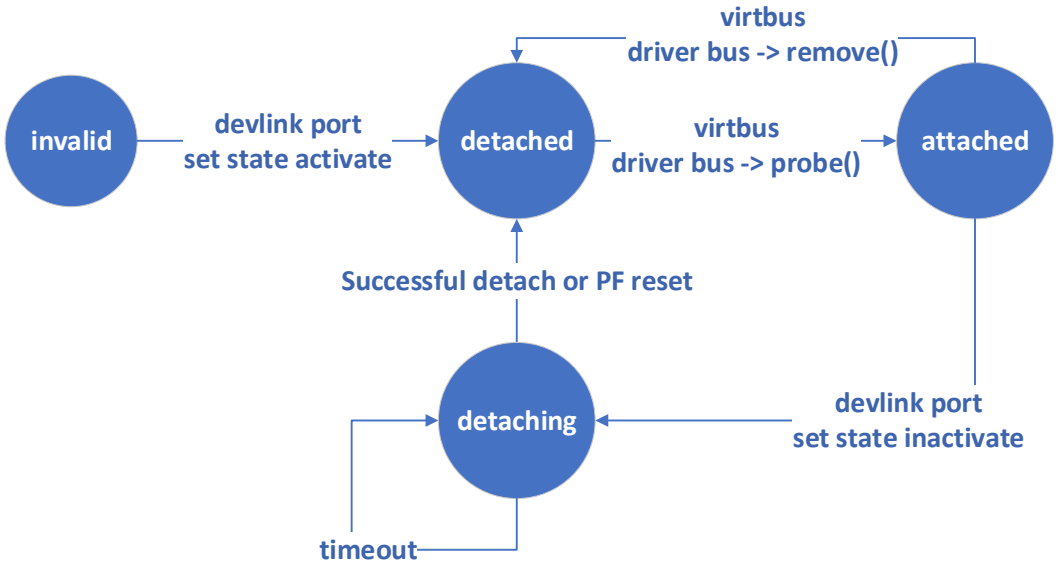
- Port state (active or inactive)
- Can be changed even when host system is power down.
- Host queries during driver load for missed events.
- Port state change trigger hot plug event on host system.
  - Triggers virtbus probe/remove.
- port state is different from driver's operational state.

NVIDIA CONFIDENTIAL. DO NOT DISTRIBUTE.

NVIDIA.

PORT FUNCTION OPERATIONAL STATES

port function operational state(driver state) machine



- Mainly driven by the driver which uses the subfunction (once activated/inactivated).
- Updated mainly for graceful plug/unplug.
  - User should wait until state changes from detaching to detached.
- Port state change trigger hot plug event on host system.
  - Triggers virtbus probe/remove.

RFC LINKS

upstream reviewed rfc links

(1) Base RFC

https://lore.kernel.org/netdev/20200519092258.GF4655@nanopsycho/

(2) RFC extension details

https://marc.info/?l=linux-netdev&m=158555928517777&w=2

(3) Port function

https://lore.kernel.org/netdev/0ac7f838-c04b-f753-2d0a-62de6a06cf9b@gmail.com/

PENDING RFC/OPEN ISSUES

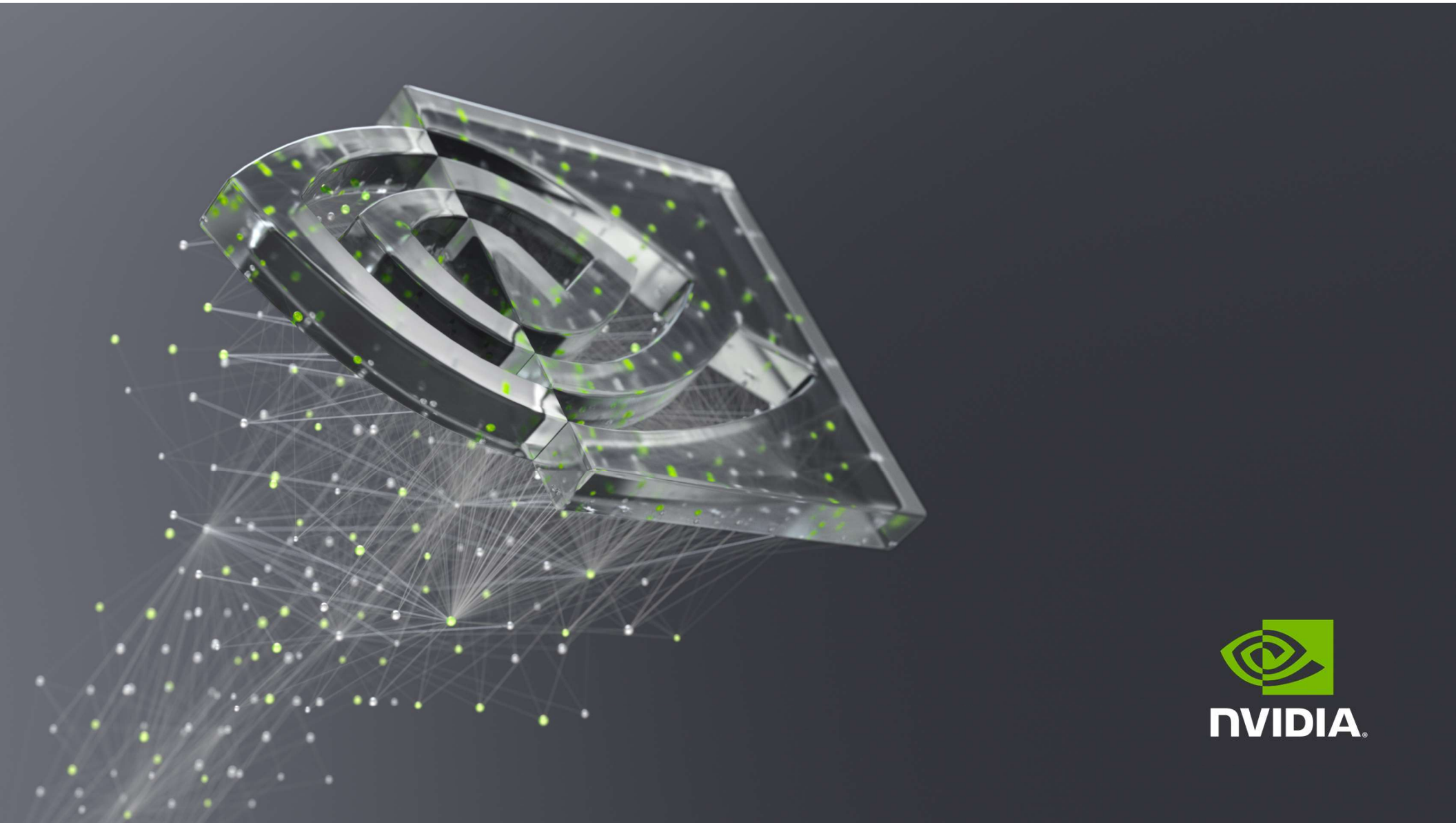
1. Host annotation for multiple hosts at devlink level
2. virtbus/ancillary bus merge to kernel -> sw bus
3. Ability to match provisioning flow for VF (like SF deploy flow)
4. iommu support to virtbus in future if needed?
5. Enable/disable only selected devices (rdma/netdev/vdpa) device of a VF/SF.

NVIDIA CONFIDENTIAL. DO NOT DISTRIBUTE.

NVIDIA.

QUESTIONS/COMMENTS?

Please send questions/comments/feedback to parav@nvidia.com


NVIDIA

BACKUP INFO

DEVICES VIEW

